

LangGraph Mastery Guide

Phase 1: Foundation

Tech Stack: Node.js · TypeScript · OpenAI

Covers: Units 0, 1, and 2 — Why LangGraph · State & Schema · Nodes & Edges

Level: Complete Beginner → Intermediate

Table of Contents

1. [Prerequisites & Environment Setup](#)
 2. [Chapter 1 — Why LangGraph](#)
 - 1.1 The Problem: How AI Workflows Used to Work
 - 1.2 LangChain Chains: What They Are and Why They Fall Short
 - 1.3 The AgentExecutor: Power but No Visibility
 - 1.4 LangGraph’s Core Idea: State Machines as Workflows
 - 1.5 When to Use What
 - 1.6 Lab 1-A: Install LangGraph and Run Your First Graph
 - 1.7 Lab 1-B: Rebuild a Chain as a LangGraph
 - 1.8 Lab 1-C: Visibility Comparison
 - 1.9 Knowledge Check
 3. [Chapter 2 — State & Schema](#)
 - 2.1 What is “State” in LangGraph?
 - 2.2 Defining State with the Annotation API
 - 2.3 Reducers: The Merge Logic Behind State Updates
 - 2.4 MessagesState: The Built-in Chat State
 - 2.5 Channels: How State Flows Between Nodes
 - 2.6 Lab 2-A: Custom State Schema
 - 2.7 Lab 2-B: Custom Reducer Functions
 - 2.8 Lab 2-C: MessagesState Chatbot
 - 2.9 Lab 2-D: Observing Merge Behavior
 - 2.10 Knowledge Check
 4. [Chapter 3 — Nodes & Edges](#)
 - 3.1 StateGraph: The Container for Your Workflow
 - 3.2 Nodes: The Workers in Your Graph
 - 3.3 Edges: The Roads Between Nodes
 - 3.4 START and END: The Entry and Exit Points
 - 3.5 Compiling the Graph
 - 3.6 Running the Graph: invoke(), stream(), and batch()
 - 3.7 Lab 3-A: Build a 3-Node Linear Graph
 - 3.8 Lab 3-B: Partial State Updates
 - 3.9 Lab 3-C: invoke() and Inspect Final State
 - 3.10 Lab 3-D: stream() Intermediate States
 - 3.11 Knowledge Check
 5. [Phase 1 Capstone Project](#)
-

Prerequisites & Environment Setup

Before we write a single line of LangGraph code, we need a working environment. This section walks you through every step — do not skip it even if you already have Node.js installed.

What You Need to Have Installed

- **Node.js** version 18 or higher (LangGraph JS requires it)
- **npm** (comes bundled with Node.js)
- A code editor — **VS Code** is recommended
- An **OpenAI API key** (get one at <https://platform.openai.com>)

Step 1 — Verify or Install Node.js

Open your terminal and run:

```
node --version
```

If you see something like `v20.11.0` — great, you are good to go. If you see an error or a version below 18, go to <https://nodejs.org> and download the LTS release.

Step 2 — Create Your Project Folder

We will create a dedicated folder for all Phase 1 work. Every chapter and lab lives inside this folder.

```
mkdir langgraph-phase1
cd langgraph-phase1
```

Step 3 — Initialise a Node.js Project

```
npm init -y
```

This creates a `package.json` file, which tracks your project dependencies. The `-y` flag accepts all defaults so you do not have to answer prompts.

Step 4 — Install TypeScript and the TypeScript Runner

```
npm install --save-dev typescript tsx @types/node
```

What each package does: - `typescript` — the TypeScript compiler that checks your code for type errors - `tsx` — a zero-config TypeScript runner that executes `.ts` files directly (no separate compile step needed during development) - `@types/node` — TypeScript type definitions for Node.js built-ins like `process`, `fs`, etc.

Step 5 — Create a TypeScript Configuration File

```
npx tsc --init
```

Then open the generated `tsconfig.json` and replace its contents with the following. Do not worry about every option — the comments explain what matters.

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "CommonJS",
    "lib": ["ES2022"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules", "dist"]
}
```

Key settings: - `"target": "ES2022"` — compile to modern JavaScript that Node.js 18+ understands - `"strict": true` — enables strict type checking, which catches bugs early - `"esModuleInterop": true` — allows `import` of CommonJS modules without workarounds

Step 6 — Install LangGraph and LangChain Packages

```
npm install @langchain/langgraph @langchain/core @langchain/openai
```

What each package does: - `@langchain/langgraph` — the core LangGraph library (state graphs, nodes, edges) - `@langchain/core` — shared primitives used across all LangChain/LangGraph packages (messages, prompts, runnables) - `@langchain/openai` — the OpenAI integration, so we can call GPT models from our graphs

Step 7 — Create Your Source Directory

```
mkdir src
```

Step 8 — Store Your OpenAI API Key

Never hard-code API keys in your source files. Instead, create a `.env` file at the project root:

```
touch .env
```

Open it and add:

```
OPENAI_API_KEY=sk-your-actual-key-here
```

Then install `dotenv` to load this file automatically:

```
npm install dotenv
```

Step 9 — Add a Run Script

Open `package.json` and add a `"scripts"` section so you can run files easily:

```
{
  "scripts": {
    "run": "tsx"
  }
}
```

Now you can run any TypeScript file with:

```
npm run run src/whatever.ts
```

Step 10 — Verify Everything Works

Create `src/hello.ts`:

```
import * as dotenv from "dotenv";
dotenv.config();

console.log("Node version:", process.version);
console.log("OpenAI key loaded:", !!process.env.OPENAI_API_KEY);
```

Run it:

```
npm run run src/hello.ts
```

Expected output:

```
Node version: v20.11.0
OpenAI key loaded: true
```

If `OpenAI key loaded: false`, double-check your `.env` file and make sure there are no spaces around the `=` sign.

Your environment is ready. Let us begin.

Chapter 1 — Why LangGraph

1.1 The Problem: How AI Workflows Used to Work

Imagine you are building a customer support bot. When a user sends a message, you want the bot to:

1. Classify whether the message is a question, complaint, or refund request
2. If it is a question — search a knowledge base and answer it
3. If it is a complaint — route it to a human agent
4. If it is a refund request — check order status, then either approve or escalate

This is a workflow. It has **branches** (different paths based on decisions), **loops** (retrying if something fails), and **state** (remembering what happened in step 1 when you are in step 4).

Before LangGraph, the two main tools for building this with LangChain were **Chains** and **AgentExecutor**. Both worked for simple cases — but both had significant limitations once workflows got complex. Understanding these limitations is the best way to appreciate why LangGraph was built.

1.2 LangChain Chains: What They Are and Why They Fall Short

What is a Chain?

A Chain is the simplest way to connect AI components in sequence. Think of it as an assembly line: input goes in one end, passes through a fixed series of steps, and output comes out the other end.

```
Input → Step 1 → Step 2 → Step 3 → Output
```

Here is a concrete example of a Chain in LangChain:

```
User message → Prompt template → LLM call → Output parser → Final answer
```

Every time the user sends a message, it flows left to right through the exact same steps in the exact same order. This is called **linear execution**.

Why linear execution is limiting:

1. **No cycles.** Once data passes through a step, it can never go back to an earlier step. If step 3 produces a bad result that needs to be regenerated, you cannot loop back to step 1. You have to run the entire chain again from scratch.
2. **No dynamic routing.** The path is fixed at design time. You cannot say “go to step B instead of step C based on what step A returned.” There are no branches.
3. **No conditional logic.** If the LLM in step 2 says “I need more information before I can answer”, the Chain has no way to act on that signal and do something different.
4. **No persistent state.** A Chain does not remember what happened in earlier steps in a structured way that later steps can inspect and modify.

The assembly line analogy: A car factory assembly line is great if every car is identical. But if a car needs extra inspection at step 5 because something unusual happened at step 3, a rigid linear line cannot handle that.

1.3 The AgentExecutor: Power but No Visibility

What is an AgentExecutor?

LangChain’s AgentExecutor was the answer to the “linear chains are too rigid” problem. An AgentExecutor gives the LLM a set of **tools** (like a web search tool, a calculator, a database lookup), and then lets the LLM decide which tools to call and in what order.

The loop looks like this:

```
User input
↓
LLM decides: "I need to search the web"
↓
Web search tool runs
↓
LLM sees the result and decides: "Now I need to calculate something"
↓
Calculator tool runs
```

```
↓
LLM decides: "I have enough information to answer"
↓
Final answer
```

This is much more powerful than a Chain because the LLM is dynamically choosing its own path. But it introduced a new set of problems:

1. **Black-box loop.** The AgentExecutor runs in a single internal loop that you have very little control over. You cannot pause it, inspect it mid-run, or intervene.
2. **Hard to debug.** If the agent gets stuck calling the same tool repeatedly, or makes a wrong decision in step 3, it is very difficult to see exactly what happened and where things went wrong.
3. **No human intervention.** The loop runs completely automatically. You cannot pause it to ask a human “does this look right before we proceed?”
4. **No control over routing.** The LLM decides everything. If you want to guarantee that certain paths are always followed, or that certain tools are only available in certain situations, that is very difficult to enforce.
5. **No persistent memory across sessions.** The AgentExecutor did not have built-in mechanisms to remember state from one conversation to the next.

The analogy: The AgentExecutor is like hiring a very smart employee and telling them “here are your tools, figure it out.” That works until they make a mistake at 2am and nobody can see what they are doing or stop them.

1.4 LangGraph’s Core Idea: State Machines as Workflows

LangGraph solves both problems by modeling AI workflows as **state machines**.

What is a state machine?

A state machine is a system that: - Has a **state** — a snapshot of all relevant information at a given moment - Has **nodes** — individual steps that can read the state and produce changes to it - Has **edges** — connections that define which node runs after which - Transitions from one node to another based on the current state

You have been using state machines your whole life without knowing it. A traffic light is a state machine: - **States:** Red, Yellow, Green - **Transitions:** Green → Yellow after 30 seconds, Yellow → Red after 5 seconds, Red → Green after 45 seconds

A washing machine is a state machine: - **States:** Filling, Washing, Rinsing, Spinning, Done - **Transitions:** Move to next state when the current cycle completes

LangGraph applies the same idea to AI workflows. Your entire workflow is represented as a **directed graph** — a diagram with boxes (nodes) and arrows (edges). Data flows through the graph as structured state.

The key advantages this gives you:

1. **Cycles are first-class.** You can have edges that loop back to earlier nodes. “Generate → Evaluate → if bad, go back to Generate” is a completely normal graph.
2. **Conditional branching.** Edges can be conditional. “After node A, go to node B if condition is met, otherwise go to node C.”
3. **Full visibility.** Because the workflow is an explicit graph, you can see exactly which node is running, what the state looks like at each step, and step through execution for debugging.
4. **Human-in-the-loop.** You can pause a graph mid-run, examine the state, modify it, get human approval, and then resume.
5. **Persistence.** The state can be saved to a database at each step, so if something crashes, execution can resume exactly where it left off.
6. **Modularity.** Graphs can be nested. You can build a sub-graph for “research” and plug it into a larger graph as a single node.

The mental model to remember:

Chain = assembly line (rigid, linear, fast to build)

AgentExecutor = autonomous employee (flexible but opaque)

LangGraph = programmable flowchart (flexible AND transparent AND controllable)

1.5 When to Use What

This is a practical decision you will make on every project:

Scenario	Best Choice	Why
Single LLM call with fixed prompt	Plain ChatOpenAI	No orchestration needed
Linear pipeline: prompt → LLM → parser	LangChain Chain (LCEL)	Simple, fast to build
Agent that needs tools, simple loop	LangChain AgentExecutor or createReactAgent	Quick to set up, sufficient for many tasks
Workflow with branches or loops	LangGraph	Conditional routing needed
Workflow needing human approval	LangGraph	Pause/resume capability
Multi-step workflow spanning multiple sessions	LangGraph + Checkpointer	Persistent state across runs
Multiple cooperating agents	LangGraph Multi-Agent	Built for this case
You need to debug step-by-step	LangGraph	Full visibility

The rule of thumb: Start with the simplest tool that solves your problem. Reach for LangGraph when you need branching, loops, persistence, or human-in-the-loop.

1.6 Lab 1-A: Install LangGraph and Run Your First One-Node Graph

Let us write the simplest possible LangGraph program. One node, one edge, done. The goal is to confirm everything is working and to see the bare minimum structure of a graph.

Create the file:

```
mkdir -p src/chapter1
touch src/chapter1/lab1a-first-graph.ts
```

Write the code:

```
// src/chapter1/lab1a-first-graph.ts

import * as dotenv from "dotenv";
dotenv.config();

// Step 1: Import the core building blocks from LangGraph
// - StateGraph: the container that holds your graph definition
// - START: a special virtual node representing where execution begins
// - END: a special virtual node representing where execution ends
// - Annotation: the API for defining your state schema
import { StateGraph, START, END, Annotation } from "@langchain/langgraph";

// =====
// STEP 2: DEFINE THE STATE SCHEMA
//
// "State" is the shared data that all nodes in your graph can read and write.
// Think of it as a whiteboard that everyone in the team can see and update.
//
// Annotation.Root() creates a type-safe state definition.
// Each field inside gets its own Annotation<T>() with:
//   - The TypeScript type (e.g., string, number, string[])
//   - A reducer function (how to merge updates into the existing value)
//   - A default function (the starting value when no input is provided)
// =====
const GraphState = Annotation.Root({
  // A simple string field that just gets overwritten on each update.
  // The reducer (existing, update) => update means:
  //   "ignore the old value, take the new value"
  // This is called "last-write-wins".
  message: Annotation<string>({
    reducer: (existing: string, update: string) => update,
    default: () => "",
  }),
});

// =====
// STEP 3: DEFINE A NODE FUNCTION
//
```

```
// A node is just a regular async function that:
// 1. Receives the current state as its argument
// 2. Does some work (can be anything: call an LLM, transform data, etc.)
// 3. Returns a PARTIAL state update (only the fields it wants to change)
//
// It does NOT need to return the entire state – just the fields that changed.
// LangGraph will merge this partial return into the full state using reducers.
//
async function greetingNode(
  state: typeof GraphState.State // TypeScript knows the exact shape of state
): Promise<Partial<typeof GraphState.State>> {

  console.log("\n[greetingNode] Running...");
  console.log("[greetingNode] Current state:", state);

  // Simulate some work – in a real graph this could be an LLM call,
  // a database lookup, an API call, etc.
  const greeting = `Hello! You said: "${state.message}". Welcome to LangGraph!`;

  console.log("[greetingNode] Returning update:", { message: greeting });

  // Return ONLY the fields that changed.
  // LangGraph merges this into the full state.
  return { message: greeting };
}

//
// STEP 4: BUILD THE GRAPH
//
// new StateGraph(GraphState) – create a graph with our state schema
// .addNode("name", function) – register a node with a name and its function
// .addEdge(from, to) – add a directed connection between two nodes
// START is the virtual "entry point" – execution always begins here
// END is the virtual "exit point" – execution terminates here
// .compile() – lock in the graph definition and prepare it
// for execution. After compile(), you cannot
// add more nodes or edges.
//
const graph = new StateGraph(GraphState)
  .addNode("greetingNode", greetingNode)
  .addEdge(START, "greetingNode") // When graph starts → run greetingNode
  .addEdge("greetingNode", END) // When greetingNode finishes → end the graph
  .compile();

//
// STEP 5: RUN THE GRAPH
//
// graph.invoke(initialState) runs the graph and returns the final state.
// The initial state you pass here will be MERGED with the defaults defined
// in your Annotation.Root(). So you only need to provide non-default values.
//
async function main() {
  console.log("=".repeat(60));
  console.log("Running Lab 1-A: First LangGraph Graph");
  console.log("=".repeat(60));

  const initialInput = { message: "I am learning LangGraph" };
  console.log("\nInitial input:", initialInput);

  const finalState = await graph.invoke(initialInput);

  console.log("\n" + "=".repeat(60));
  console.log("Final state:", finalState);
  console.log("=".repeat(60));
}

main().catch(console.error);
```

Run it:

```
npm run run src/chapter1/lab1a-first-graph.ts
```

Expected output:

```
=====
Running Lab 1-A: First LangGraph Graph
=====

Initial input: { message: 'I am learning LangGraph' }

[greetingNode] Running...
[greetingNode] Current state: { message: 'I am learning LangGraph' }
[greetingNode] Returning update: { message: "Hello! You said: \"I am learning LangGraph\". Welcome to LangGraph!" }

=====
Final state: { message: "Hello! You said: \"I am learning LangGraph\". Welcome to LangGraph!" }
```

=====

How to verify it worked: The final state's `message` field should contain the transformed greeting, not the original input. The node successfully read state, transformed it, and returned a partial update that LangGraph merged back into state.

Common beginner mistakes:

- *Forgetting `dotenv.config()` at the top:* Your `OPENAI_API_KEY` will not load. Always put this at the very first line of your main file.
- *Returning the full state from a node:* You only need to return the fields that changed. Returning extra fields is fine but unnecessary.
- *Forgetting `.compile()`:* The graph will throw an error if you try to invoke it without compiling first.

1.7 Lab 1-B: Rebuild a Simple Chain as a LangGraph Graph

Now we will do something more meaningful — use an actual LLM. We will first see what a two-step Chain would look like conceptually, then build the equivalent as a LangGraph.

The workflow: “Translate a sentence to French, then summarize it in one word.”

Create the file:

```
touch src/chapter1/lab1b-chain-to-graph.ts
```

```
// src/chapter1/lab1b-chain-to-graph.ts

import * as dotenv from "dotenv";
dotenv.config();

import { StateGraph, START, END, Annotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

// -----
// INITIALISE THE LLM
//
// ChatOpenAI wraps OpenAI's chat completion API.
// - model: which model to use (gpt-4o-mini is fast and affordable)
// - temperature: 0 = deterministic/consistent, 1 = more creative/random
// -----
const llm = new ChatOpenAI({
  model: "gpt-4o-mini",
  temperature: 0,
});

// -----
// STATE SCHEMA
//
// This time our state has three fields:
//   originalText – the text the user wants to process
//   translatedText – after node 1 runs (French translation)
//   summary – after node 2 runs (one-word summary)
//
// Note: all reducers here are "last-write-wins" (update overwrites existing).
// This is appropriate for fields where only the latest value matters.
// -----
const PipelineState = Annotation.Root({
  originalText: Annotation<string>({
    reducer: (_existing: string, update: string) => update,
    default: () => "",
  }),
  translatedText: Annotation<string>({
    reducer: (_existing: string, update: string) => update,
    default: () => "",
  }),
  summary: Annotation<string>({
    reducer: (_existing: string, update: string) => update,
    default: () => "",
  }),
});

// -----
// NODE 1: TRANSLATE TO FRENCH
//
// Reads: state.originalText
// Writes: state.translatedText
// -----
async function translateToFrench(
  state: typeof PipelineState.State
): Promise<Partial<typeof PipelineState.State>> {
```



```

console.log("\n[translateToFrench] Input:", state.originalText);

const response = await llm.invoke([
  new SystemMessage("You are a professional translator. Translate the given text to French. Return ONLY the translation, nothing else."),
  new HumanMessage(state.originalText),
]);

// response.content contains the LLM's reply as a string
const translation = response.content as string;
console.log("[translateToFrench] Result:", translation);

return { translatedText: translation };
}

// =====
// NODE 2: SUMMARIZE IN ONE WORD
//
// Reads: state.translatedText (the output from Node 1)
// Writes: state.summary
//
// Notice: this node uses the result of Node 1 naturally, just by reading
// from state. There is no function call linking Node 1 to Node 2 explicitly –
// the graph structure handles that via edges.
// =====
async function summarizeInOneWord(
  state: typeof PipelineState.State
): Promise<Partial<typeof PipelineState.State>> {

  console.log("\n[summarizeInOneWord] Input:", state.translatedText);

  const response = await llm.invoke([
    new SystemMessage("You are a text summarizer. Summarize the given text in exactly ONE word. Return ONLY that one word."),
    new HumanMessage(state.translatedText),
  ]);

  const summary = response.content as string;
  console.log("[summarizeInOneWord] Result:", summary);

  return { summary };
}

// =====
// BUILD THE GRAPH
//
// The graph models the two-step pipeline:
// START → translateToFrench → summarizeInOneWord → END
//
// Compare this to a Chain: the logic is identical, but as a graph you can
// later add branches, loops, and human checkpoints without restructuring.
// =====
const pipeline = new StateGraph(PipelineState)
  .addNode("translate", translateToFrench)
  .addNode("summarize", summarizeInOneWord)
  .addEdge(START, "translate") // 1. Start → Translate
  .addEdge("translate", "summarize") // 2. Translate → Summarize
  .addEdge("summarize", END) // 3. Summarize → Done
  .compile();

async function main() {
  console.log("=".repeat(60));
  console.log("Lab 1-B: Two-Node LangGraph Pipeline (with LLM)");
  console.log("=".repeat(60));

  const result = await pipeline.invoke({
    originalText: "Learning new things every day makes life more interesting.",
  });

  console.log("\n" + "=".repeat(60));
  console.log("PIPELINE RESULTS:");
  console.log("Original   :", result.originalText);
  console.log("Translated:", result.translatedText);
  console.log("Summary    :", result.summary);
  console.log("=".repeat(60));
}

main().catch(console.error);

```

Run it:

```
npm run run src/chapter1/lab1b-chain-to-graph.ts
```

Expected output:

```

=====
Lab 1-B: Two-Node LangGraph Pipeline (with LLM)

```

```
=====

[translateToFrench] Input: Learning new things every day makes life more interesting.
[translateToFrench] Result: Apprendre de nouvelles choses chaque jour rend la vie plus intéressante.

[summarizeInOneWord] Input: Apprendre de nouvelles choses chaque jour rend la vie plus intéressante.
[summarizeInOneWord] Result: Enrichissement

=====

PIPELINE RESULTS:
Original   : Learning new things every day makes life more interesting.
Translated: Apprendre de nouvelles choses chaque jour rend la vie plus intéressante.
Summary    : Enrichissement
=====
```

Key insight: Notice how `summarizeInOneWord` naturally received the output of `translateToFrench` — not because we passed it explicitly, but because they both share the same `state` object and the graph runs them in sequence. The state is the communication channel between nodes.

1.8 Lab 1-C: Visibility Comparison — Seeing Each Step

One of LangGraph’s biggest advantages over AgentExecutor is that you can see every intermediate state. Let us demonstrate this by streaming the graph’s output step by step.

Create the file:

```
touch src/chapter1/lab1c-visibility.ts
```

```
// src/chapter1/lab1c-visibility.ts

import * as dotenv from "dotenv";
dotenv.config();

import { StateGraph, START, END, Annotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

const State = Annotation.Root({
  topic: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "",
  }),
  outline: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "",
  }),
  draft: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "",
  }),
  finalArticle: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "",
  }),
});

async function createOutline(state: typeof State.State) {
  const res = await llm.invoke([
    new SystemMessage("Create a 3-point outline for a short article. Return ONLY the numbered outline."),
    new HumanMessage(`Topic: ${state.topic}`),
  ]);
  return { outline: res.content as string };
}

async function writeDraft(state: typeof State.State) {
  const res = await llm.invoke([
    new SystemMessage("Write a short 2-sentence article based on this outline. Be concise."),
    new HumanMessage(`Outline:\n${state.outline}`),
  ]);
  return { draft: res.content as string };
}

async function polish(state: typeof State.State) {
  const res = await llm.invoke([
    new SystemMessage("Polish this draft. Fix any grammar issues and make it flow better. Return the improved text only."),
    new HumanMessage(state.draft),
  ]);
  return { finalArticle: res.content as string };
}
```

```

const articleGraph = new StateGraph(State)
  .addNode("outline", createOutline)
  .addNode("draft", writeDraft)
  .addNode("polish", polish)
  .addEdge(START, "outline")
  .addEdge("outline", "draft")
  .addEdge("draft", "polish")
  .addEdge("polish", END)
  .compile();

async function main() {
  console.log("=".repeat(60));
  console.log("Lab 1-C: Streaming Graph Execution (Full Visibility)");
  console.log("=".repeat(60));

  // -----
  // graph.stream() returns an async iterator.
  // Each iteration gives you ONE node's output – the partial state update
  // that node returned.
  //
  // This is the key visibility advantage: with a Chain, you only see the
  // final result. With LangGraph streaming, you see each intermediate step
  // as it happens, in real time.
  // -----
  const stream = await articleGraph.stream({
    topic: "The benefits of learning programming",
  });

  let stepNumber = 0;
  for await (const chunk of stream) {
    stepNumber++;
    // Each chunk is an object like: { nodeName: { field: value } }
    // The outer key is the name of the node that just ran.
    const [nodeName, nodeOutput] = Object.entries(chunk)[0];

    console.log(`\n${"=".repeat(50)}`);
    console.log(`STEP ${stepNumber} – Node: "${nodeName}" completed`);
    console.log(`${"=".repeat(50)}`);
    console.log("Output from this node:", JSON.stringify(nodeOutput, null, 2));
  }

  console.log("\n" + "=".repeat(60));
  console.log("All nodes completed. Graph execution finished.");
  console.log("=".repeat(60));
}

main().catch(console.error);

```

Run it:

```
npm run run src/chapter1/lab1c-visibility.ts
```

You should see three separate blocks of output, one for each node, appearing one after another as the graph progresses. This real-time visibility into each step is the core difference from a Chain (which would only show the final output).

1.9 Knowledge Check — Chapter 1

Summary

- **Chains** are linear pipelines. They are great for simple, fixed workflows but cannot branch or loop.
- **AgentExecutor** is more powerful but operates as a black box — hard to debug, impossible to pause.
- **LangGraph** models workflows as state machines — directed graphs with shared state flowing between nodes.
- LangGraph gives you branching, cycles, full visibility, human-in-the-loop capability, and persistence.
- Use the simplest tool that solves your problem. Reach for LangGraph when complexity demands it.

Key Takeaways

1. A LangGraph program has three core concepts: **State** (the shared data), **Nodes** (the processing steps), and **Edges** (the connections between steps).
2. `StateGraph` is the container. `addNode()` registers a step. `addEdge()` connects steps. `compile()` locks it in.
3. `invoke()` runs the graph and returns the final state. `stream()` yields each node's output as it completes.
4. Nodes return **partial** state updates — only the fields that changed.

Practice Questions

1. What specific limitation of Chains does LangGraph solve by allowing cycles?

2. Why is the AgentExecutor considered a “black box”? What does that mean in practice?
3. In LangGraph, what is the difference between `invoke()` and `stream()` ?
4. What does `.compile()` do and why must it be called before `.invoke()` ?
5. Why do nodes return only partial state instead of the full state?

Mini Exercise

Extend `lab1b-chain-to-graph.ts` to add a **fourth node** that takes the one-word French summary and translates it back to English. The flow should be:

```
START → translate → summarize → translateBack → END
```

Verify that the final state has all four fields populated correctly.

Chapter 2 — State & Schema

2.1 What is “State” in LangGraph?

State is the most important concept in LangGraph. Everything else — nodes, edges, reducers — exists to read and modify state. Getting state design right is the difference between a clean graph and a confusing one.

The analogy: a whiteboard in a meeting room

Imagine your graph is a team of people working on a project in a room with a whiteboard. The whiteboard shows: - The current task - What has been completed so far - Any errors or issues found - The latest draft of the output

Every person (node) can walk up to the whiteboard, read what is on it, add new information, or update existing information. When a person finishes their work, they write their contribution on the whiteboard and sit down. The next person reads the whiteboard to see what they need to work with.

That whiteboard is the **state**. It is shared, structured, and always up to date.

Technical definition:

State in LangGraph is a plain JavaScript/TypeScript object — a dictionary of key-value pairs. Each node receives the current state and returns a partial update. LangGraph merges that partial update into the full state using **reducers** (covered in section 2.3) before passing state to the next node.

2.2 Defining State with the Annotation API

LangGraph provides the `Annotation` API to define state schemas in TypeScript with full type safety.

The pattern:

```
import { Annotation } from "@langchain/langgraph";

const MyState = Annotation.Root({
  fieldName: Annotation<FieldType>({
    reducer: (existing: FieldType, update: FieldType) => mergeLogic,
    default: () => initialValue,
  }),
  // ... more fields
});
```

Let us break down each part:

`Annotation.Root({ ... })`

This is the top-level function that creates your state type. Think of it as defining the columns of a table. Every field you define here becomes a property of your state object.

`Annotation<FieldType>({ ... })`

This defines one field. The generic type parameter `<FieldType>` tells TypeScript what kind of data this field holds. It can be: - `string` - `number` - `boolean` - `string[]` (array of strings) - `number[]` - Any custom TypeScript interface or type

reducer

A function that defines how to merge a new value into the existing value. It receives two arguments: - `existing` — the current value in state before this update - `update` — the new value the node is trying to write

The function returns the result — what the field should be after the merge.

default

A function that returns the initial value of this field when no value is provided in the initial input to `invoke()`. Using a function (instead of a plain value) ensures each graph run gets a fresh copy — especially important for arrays and objects.

Accessing the TypeScript type of your state:

After defining `MyState`, you can get the TypeScript type of the state object using:

```
typeof MyState.State
// This gives you: { fieldName: FieldType, ... }
```

Use this in node function signatures to get full type safety and autocomplete.

2.3 Reducers: The Merge Logic Behind State Updates

Reducers are the most important concept to understand clearly. They determine how state evolves as nodes run.

Why reducers exist:

When a node returns `{ message: "Hello" }`, LangGraph does not simply replace the entire state with `{ message: "Hello" }`. Instead, it calls the `reducer` function for each field that was returned, merging the new value with the existing value according to the rules you defined.

This design enables several powerful patterns:

Pattern 1: Last-Write-Wins (Replace)

The most common pattern. The new value simply replaces the old one. Use this for fields like “current answer”, “latest status”, “user input”.

```
fieldName: Annotation<string>({
  reducer: (_existing: string, update: string) => update,
  //   ^ ignore old value   ^ take new value
  default: () => "",
}),
```

The underscore prefix on `_existing` is a TypeScript convention meaning “this parameter is intentionally unused.”

Pattern 2: Accumulate (Append)

The new value is added to the existing value. Use this for fields like “message history”, “collected results”, “error log”.

```
messages: Annotation<string[]>({
  reducer: (existing: string[], update: string[]) => [...existing, ...update],
  //   ^ keep old items           ^ add new items
  default: () => [],
}),
```

Note: the spread operator `...` creates a new array combining both. This is important — avoid mutating `existing` directly.

Pattern 3: Numeric Accumulation (Add)

The new value is added arithmetically. Use this for counters, totals, retry counts.

```
retryCount: Annotation<number>({
  reducer: (existing: number, update: number) => existing + update,
  default: () => 0,
}),
```

A node would then return `{ retryCount: 1 }` to increment the counter by 1, regardless of the current value.

Pattern 4: Smart Merge (Conditional)

The reducer can contain any logic you want, including conditionals.

```
bestScore: Annotation<number>({
```

```

    reducer: (existing: number, update: number) => Math.max(existing, update),
    // Always keeps the highest score seen
    default: () => 0,
  })),
},

```

The crucial insight about reducers:

The reducer is called on the **receiving end** — when LangGraph processes a node’s return value. The node itself does not need to know what the current value is to write an update. For example, to append a message, the node just returns `{ messages: [newMessage] }`. The reducer handles adding it to the existing list.

2.4 MessagesState: The Built-in Chat State

Because building chatbots with conversation history is so common, LangGraph provides a pre-built state schema called `MessagesAnnotation`. It has exactly one field: `messages`, which is an array of LangChain message objects.

```

import { MessagesAnnotation } from "@langchain/langgraph";
import { HumanMessage, AIMessage, SystemMessage } from "@langchain/core/messages";

```

The `messages` field’s built-in reducer:

The built-in reducer for `messages` is smart. It does not just append blindly. It uses the LangChain `addMessages` utility, which: - Appends new messages to the existing list - Deduplicates based on message ID (important for streaming and re-runs)

Message types in LangChain:

Class	Role	Used by
HumanMessage	A message from the user	You when invoking the graph
AIMessage	A response from the AI	LLM nodes
SystemMessage	Instructions to the AI	Usually the first message
ToolMessage	Result from a tool call	Tool nodes

Using MessagesAnnotation:

```

import { StateGraph, START, END } from "@langchain/langgraph";
import { MessagesAnnotation } from "@langchain/langgraph";

// When using MessagesAnnotation, your graph already has a `messages` field.
// You do not need to define it yourself.

const graph = new StateGraph(MessagesAnnotation)
  .addNode("chat", async (state) => {
    // state.messages is an array of message objects
    const lastMessage = state.messages[state.messages.length - 1];
    // ... call LLM and return new message
    return { messages: [new AIMessage("My response")] };
    // The reducer appends this to the existing messages array
  })
  .addEdge(START, "chat")
  .addEdge("chat", END)
  .compile();

await graph.invoke({
  messages: [new HumanMessage("Hello!")]
});

```

2.5 Channels: How State Flows Between Nodes

You may encounter the word “channel” in LangGraph documentation. A channel is simply the internal mechanism that LangGraph uses to manage one field of state. Each field in your `Annotation.Root()` corresponds to one channel.

You do not need to interact with channels directly — the `Annotation` API abstracts this away. But understanding the concept helps explain why LangGraph is architected the way it is:

- Each channel has its own reducer (the merge function)
- Each channel has its own current value
- When a node returns an update, LangGraph distributes that update to the relevant channels, each channel applies its reducer, and the new combined state is assembled for the next node

The channel abstraction is what makes parallel node execution possible (covered in Phase 2 — Control Flow). Multiple nodes can update different channels simultaneously without interfering with each other.

2.6 Lab 2-A: Define a Custom State Schema with Multiple Typed Fields

Let us build a graph with a rich, realistic state schema.

Create the file:

```
touch src/chapter2/lab2a-custom-schema.ts
mkdir -p src/chapter2
touch src/chapter2/lab2a-custom-schema.ts
```

```
// src/chapter2/lab2a-custom-schema.ts

import * as dotenv from "dotenv";
dotenv.config();

import { StateGraph, START, END, Annotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

// -----
// A REALISTIC MULTI-FIELD STATE SCHEMA
//
// Imagine a job application screening workflow.
// The state tracks everything we know about a candidate and the process.
// -----
const ScreeningState = Annotation.Root({
  // The raw resume text – set once at the start, never changes
  resumeText: Annotation<string>({
    reducer: (_existing: string, update: string) => update, // last-write-wins
    default: () => "",
  }),

  // The job role we are screening for – set once at the start
  jobRole: Annotation<string>({
    reducer: (_existing: string, update: string) => update,
    default: () => "",
  }),

  // A list of skills extracted from the resume – we APPEND to this
  extractedSkills: Annotation<string[]>({
    reducer: (existing: string[], update: string[]) => [...existing, ...update],
    default: () => [], // start with an empty array
  }),

  // Screening score out of 100 – last-write-wins
  score: Annotation<number>({
    reducer: (_existing: number, update: number) => update,
    default: () => 0,
  }),

  // Number of processing steps completed – we INCREMENT this
  stepsCompleted: Annotation<number>({
    reducer: (existing: number, update: number) => existing + update,
    default: () => 0,
  }),

  // Whether the candidate passed screening
  passed: Annotation<boolean>({
    reducer: (_existing: boolean, update: boolean) => update,
    default: () => false,
  }),

  // A list of feedback notes added by different nodes – we APPEND to this
  feedbackNotes: Annotation<string[]>({
    reducer: (existing: string[], update: string[]) => [...existing, ...update],
    default: () => [],
  }),

  // Final decision message – last-write-wins
  decision: Annotation<string>({
    reducer: (_existing: string, update: string) => update,
    default: () => "",
  }),
});

// -----
// NODE 1: EXTRACT SKILLS FROM RESUME
// -----
```

```

async function extractSkills(
  state: typeof ScreeningState.State
): Promise<Partial<typeof ScreeningState.State>> {
  console.log("\n[extractSkills] Analysing resume...");

  const res = await llm.invoke([
    new SystemMessage(
      "Extract the top 5 technical skills from this resume as a comma-separated list. " +
      "Return ONLY the comma-separated list, nothing else. Example: Python, SQL, React, AWS, Docker"
    ),
    new HumanMessage(state.resumeText),
  ]);

  const skillsRaw = res.content as string;
  const skills = skillsRaw.split(",").map((s) => s.trim()).filter(Boolean);
  console.log("[extractSkills] Skills found:", skills);

  return {
    extractedSkills: skills, // reducer APPENDS these to existing array
    stepsCompleted: 1,      // reducer ADDS 1 to existing count
    feedbackNotes: [`Found ${skills.length} technical skills in resume`],
  };
}

// =====
// NODE 2: SCORE THE CANDIDATE
// =====
async function scoreCandidate(
  state: typeof ScreeningState.State
): Promise<Partial<typeof ScreeningState.State>> {
  console.log("\n[scoreCandidate] Scoring candidate for role:", state.jobRole);
  console.log("[scoreCandidate] Skills to evaluate:", state.extractedSkills);

  const res = await llm.invoke([
    new SystemMessage(
      "You are a technical recruiter. Based on the candidate's skills and the job role, " +
      "give a score from 0 to 100. Return ONLY the number."
    ),
    new HumanMessage(
      `Job Role: ${state.jobRole}\n` +
      `Candidate Skills: ${state.extractedSkills.join(", ")}`
    ),
  ]);

  const scoreText = res.content as string;
  const score = parseInt(scoreText.trim(), 10) || 0;
  console.log("[scoreCandidate] Score assigned:", score);

  return {
    score,
    stepsCompleted: 1, // adds 1 more (total will be 2)
    feedbackNotes: [`Fit score for ${state.jobRole}: ${score}/100`],
  };
}

// =====
// NODE 3: MAKE FINAL DECISION
// =====
async function makeDecision(
  state: typeof ScreeningState.State
): Promise<Partial<typeof ScreeningState.State>> {
  console.log("\n[makeDecision] Final score:", state.score);

  const passed = state.score >= 70;
  const decision = passed
    ? `PASS - Candidate scored ${state.score}/100. Recommend proceeding to interview.`
    : `FAIL - Candidate scored ${state.score}/100. Does not meet the 70/100 threshold.`;

  return {
    passed,
    decision,
    stepsCompleted: 1, // adds 1 more (total will be 3)
    feedbackNotes: [decision],
  };
}

// BUILD THE GRAPH
const screeningGraph = new StateGraph(ScreeningState)
  .addNode("extractSkills", extractSkills)
  .addNode("score", scoreCandidate)
  .addNode("decide", makeDecision)
  .addEdge(START, "extractSkills")
  .addEdge("extractSkills", "score")
  .addEdge("score", "decide")
  .addEdge("decide", END)
  .compile();

// =====
// RUN THE GRAPH
// =====

```



```

async function main() {
  console.log("=".repeat(60));
  console.log("Lab 2-A: Multi-Field State Schema");
  console.log("=".repeat(60));

  const sampleResume = `
    John Smith – Software Engineer
    5 years experience in full-stack web development.
    Skills: TypeScript, React, Node.js, PostgreSQL, Docker, AWS
    Led development of 3 production applications.
    Strong background in agile methodologies.
  `;

  const result = await screeningGraph.invoke({
    resumeText: sampleResume,
    jobRole: "Senior TypeScript Developer",
  });

  console.log("\n" + "=".repeat(60));
  console.log("FINAL STATE:");
  console.log("=".repeat(60));
  console.log("Extracted Skills :", result.extractedSkills);
  console.log("Score           :", result.score);
  console.log("Steps Completed  :", result.stepsCompleted);
  console.log("Passed           :", result.passed);
  console.log("Feedback Notes   :", result.feedbackNotes);
  console.log("Decision          :", result.decision);
  console.log("=".repeat(60));
}

main().catch(console.error);

```

Run it:

```
npm run run src/chapter2/lab2a-custom-schema.ts
```

What to observe: - `extractedSkills` is an array that gets skills appended, not replaced - `stepsCompleted` increments by 1 with each node — watch the final value be 3 - `feedbackNotes` collects notes from all three nodes into one array

2.7 Lab 2-B: Custom Reducer Functions for a Counter

This lab focuses specifically on understanding reducer behavior by visualising what happens step by step.

Create the file:

```
touch src/chapter2/lab2b-custom-reducers.ts
```

```

// src/chapter2/lab2b-custom-reducers.ts
// This lab uses NO LLM – purely demonstrates reducer behavior with simple data.

import { StateGraph, START, END, Annotation } from "@langchain/langgraph";

// =====
// STATE WITH THREE DIFFERENT REDUCER TYPES
// =====

const CounterState = Annotation.Root({

  // LAST-WRITE-WINS: the new value simply replaces the old one
  currentPhase: Annotation<string>({
    reducer: (_existing: string, update: string) => update,
    default: () => "not started",
  }),

  // ACCUMULATE (SUM): each update adds to the running total
  totalPoints: Annotation<number>({
    reducer: (existing: number, update: number) => existing + update,
    default: () => 0,
  }),

  // ACCUMULATE (ARRAY): each update appends items to the list
  completedTasks: Annotation<string[]>({
    reducer: (existing: string[], update: string[]) => [...existing, ...update],
    default: () => [],
  }),

  // KEEP MAX: only replaces if the new value is larger
  highScore: Annotation<number>({
    reducer: (existing: number, update: number) => Math.max(existing, update),
    default: () => 0,
  }),

```

```

// KEEP FIRST (NEVER OVERWRITE): only accepts the first non-empty value
firstReachedMilestone: Annotation<string>({
  reducer: (existing: string, update: string) => existing || update,
  // If existing is non-empty (truthy), keep it. Otherwise use update.
  default: () => "",
}),
});

// -----
// HELPER: prints a divider and the current state
// -----
function printState(label: string, state: typeof CounterState.State) {
  console.log(`\n${"─".repeat(50)}`);
  console.log(`AFTER: ${label}`);
  console.log(`${"─".repeat(50)}`);
  console.log("currentPhase      :", state.currentPhase);
  console.log("totalPoints          :", state.totalPoints);
  console.log("completedTasks       :", state.completedTasks);
  console.log("highScore            :", state.highScore);
  console.log("firstReachedMilestone :", state.firstReachedMilestone);
}

// -----
// NODES – each one deliberately returns values that test different reducers
// -----

async function phaseOne(
  state: typeof CounterState.State
): Promise<Partial<typeof CounterState.State>> {
  console.log("\n[phaseOne] Executing...");
  return {
    currentPhase: "Phase 1",      // REPLACES "not started"
    totalPoints: 10,              // ADDS 10 to 0 → total = 10
    completedTasks: ["task-A"],   // APPENDS → ["task-A"]
    highScore: 42,                // 42 > 0 → kept
    firstReachedMilestone: "Phase 1 Start", // first non-empty → kept
  };
}

async function phaseTwo(
  state: typeof CounterState.State
): Promise<Partial<typeof CounterState.State>> {
  console.log("\n[phaseTwo] Executing...");
  return {
    currentPhase: "Phase 2",      // REPLACES "Phase 1"
    totalPoints: 25,              // ADDS 25 to 10 → total = 35
    completedTasks: ["task-B", "task-C"], // APPENDS → ["task-A", "task-B", "task-C"]
    highScore: 30,                // 30 < 42 → NOT replaced, stays 42
    firstReachedMilestone: "Phase 2 Start", // existing is non-empty → IGNORED
  };
}

async function phaseThree(
  state: typeof CounterState.State
): Promise<Partial<typeof CounterState.State>> {
  console.log("\n[phaseThree] Executing...");
  return {
    currentPhase: "Complete",     // REPLACES "Phase 2"
    totalPoints: 50,              // ADDS 50 to 35 → total = 85
    completedTasks: ["task-D"],   // APPENDS → ["task-A", "task-B", "task-C", "task-D"]
    highScore: 99,                // 99 > 42 → replaces, now 99
    firstReachedMilestone: "Phase 3 Start", // existing is non-empty → IGNORED
  };
}

const graph = new StateGraph(CounterState)
  .addNode("phaseOne", phaseOne)
  .addNode("phaseTwo", phaseTwo)
  .addNode("phaseThree", phaseThree)
  .addEdge(START, "phaseOne")
  .addEdge("phaseOne", "phaseTwo")
  .addEdge("phaseTwo", "phaseThree")
  .addEdge("phaseThree", END)
  .compile();

async function main() {
  console.log("=".repeat(60));
  console.log("Lab 2-B: Custom Reducers – Step-by-Step Observation");
  console.log("=".repeat(60));
  console.log("\nInitial state (all defaults):");
  console.log("currentPhase: 'not started'");
  console.log("totalPoints: 0");
  console.log("completedTasks: []");
  console.log("highScore: 0");
  console.log("firstReachedMilestone: ''");

  // Use stream() to see state after each node
  const stream = await graph.stream({});

  for await (const chunk of stream) {

```

```

    const [nodeName, _output] = Object.entries(chunk)[0];
    // To see state AFTER each node, we need to use streamMode: "values"
    // The default stream mode returns node outputs (partial updates)
    // We will cover streamMode in Chapter 3. For now, log the partial output.
    console.log(`\n[${nodeName}] returned partial update:`, chunk[nodeName]);
  }

  // Invoke separately to get the final merged state
  const finalState = await graph.invoke({});
  printState("All nodes complete", finalState);

  console.log("\n" + "=".repeat(60));
  console.log("REDUCER BEHAVIOR SUMMARY:");
  console.log("-".repeat(60));
  console.log("currentPhase      : last-write-wins → final = 'Complete'");
  console.log("totalPoints          : sum → 0 + 10 + 25 + 50 = 85");
  console.log("completedTasks       : append → 4 tasks total");
  console.log("highScore            : keep-max → 42 → 42 (30 lost) → 99");
  console.log("firstReachedMilestone: keep-first → 'Phase 1 Start' (unchanged)");
  console.log("=".repeat(60));
}

main().catch(console.error);

```

Run it:

```
npm run run src/chapter2/lab2b-custom-reducers.ts
```

Verify the final values match the expected reducer behavior described in the summary section.

2.8 Lab 2-C: MessagesState Chatbot

Let us build a real multi-turn chatbot using the built-in `MessagesAnnotation`.

Create the file:

```
touch src/chapter2/lab2c-messages-state.ts
```

```

// src/chapter2/lab2c-messages-state.ts

import * as dotenv from "dotenv";
dotenv.config();

import { StateGraph, START, END } from "@langchain/langgraph";
import { MessagesAnnotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0.7 });

// -----
// THE CHAT NODE
//
// MessagesAnnotation.State has one field: messages (BaseMessage[])
// The node reads the full conversation history and calls the LLM with it.
// The LLM's response is returned as a new message to be appended.
// -----
async function chatNode(
  state: typeof MessagesAnnotation.State
): Promise<Partial<typeof MessagesAnnotation.State>> {
  console.log(`\n[chatNode] Conversation has ${state.messages.length} message(s)`);

  // Pass the ENTIRE conversation history to the LLM.
  // This is how the bot remembers what was said earlier.
  // We prepend a system message to give the bot its persona.
  const response = await llm.invoke([
    new SystemMessage(
      "You are a friendly and concise assistant named Max. " +
      "Keep responses to 2-3 sentences maximum."
    ),
    ...state.messages, // spread all existing messages
  ]);

  console.log("[chatNode] Bot response:", response.content);

  // Return the AI response – the reducer appends it to the messages array
  return { messages: [response] };
}

const chatGraph = new StateGraph(MessagesAnnotation)
  .addNode("chat", chatNode)

```

```

.addEdge(START, "chat")
.addEdge("chat", END)
.compile();

// -----
// SIMULATE A MULTI-TURN CONVERSATION
//
// Each call to graph.invoke() is a new turn.
// We manually maintain the conversation history between turns by
// passing the accumulated messages in each call.
//
// In Phase 3 (Persistence), you will learn how to use Checkpointers
// so that LangGraph stores conversation history automatically.
// -----
async function main() {
  console.log("=".repeat(60));
  console.log("Lab 2-C: Multi-Turn Chatbot with MessagesAnnotation");
  console.log("=".repeat(60));

  // Turn 1
  console.log("\n--- TURN 1 ---");
  const turn1 = await chatGraph.invoke({
    messages: [new HumanMessage("Hi Max! What is your favourite programming language?")],
  });
  console.log("Messages after turn 1:", turn1.messages.length, "message(s)");

  // Turn 2 – pass ALL previous messages so the bot remembers context
  console.log("\n--- TURN 2 ---");
  const turn2 = await chatGraph.invoke({
    messages: [
      ...turn1.messages, // include turn 1 history
      new HumanMessage("Why do you like it so much?"),
    ],
  });
  console.log("Messages after turn 2:", turn2.messages.length, "message(s)");

  // Turn 3 – test if the bot remembers what it said in turn 1
  console.log("\n--- TURN 3 ---");
  const turn3 = await chatGraph.invoke({
    messages: [
      ...turn2.messages, // include full history
      new HumanMessage("What was the name of the language you mentioned?"),
    ],
  });
  console.log("Messages after turn 3:", turn3.messages.length, "message(s)");

  // Print full conversation
  console.log("\n" + "=".repeat(60));
  console.log("FULL CONVERSATION:");
  console.log("=".repeat(60));
  for (const msg of turn3.messages) {
    const role = msg._getType() === "human" ? "You " : "Max ";
    console.log(`${role}: ${msg.content}`);
  }
}

main().catch(console.error);

```

Run it:

```
npm run run src/chapter2/lab2c-messages-state.ts
```

What to observe: In Turn 3, the bot correctly remembers the language name it mentioned in Turn 1 — because the full conversation history is passed in `state.messages`. This demonstrates how `MessagesAnnotation` accumulates context across turns.

2.9 Lab 2-D: Observing How Two Nodes Update the Same Key

This short lab demonstrates what happens when two different nodes both try to write to the same state field.

Create the file:

```
touch src/chapter2/lab2d-merge-behavior.ts
```

```

// src/chapter2/lab2d-merge-behavior.ts
// No LLM needed – demonstrates reducer merge behavior with pure data

import { StateGraph, START, END, Annotation } from "@langchain/langgraph";

const MergeState = Annotation.Root({
  // REPLACE reducer: whichever node runs LAST wins

```

```

latestStatus: Annotation<string>({
  reducer: (_e: string, u: string) => u,
  default: () => "pending",
}),

// APPEND reducer: both nodes contribute, order matters
log: Annotation<string[]>({
  reducer: (e: string[], u: string[]) => [...e, ...u],
  default: () => [],
}),

// SUM reducer: both contributions are added together
totalVotes: Annotation<number>({
  reducer: (e: number, u: number) => e + u,
  default: () => 0,
}),
});

async function nodeA(state: typeof MergeState.State) {
  console.log("[nodeA] Running. Current latestStatus:", state.latestStatus);
  return {
    latestStatus: "updated by A",
    log: ["nodeA ran"],
    totalVotes: 3,
  };
}

async function nodeB(state: typeof MergeState.State) {
  // nodeB sees the state AFTER nodeA has run and been merged
  console.log("[nodeB] Running. Current latestStatus:", state.latestStatus);
  return {
    latestStatus: "updated by B", // this will overwrite A's value
    log: ["nodeB ran"],          // this will APPEND after A's entry
    totalVotes: 7,               // this will ADD to A's 3 → total 10
  };
}

const graph = new StateGraph(MergeState)
  .addNode("nodeA", nodeA)
  .addNode("nodeB", nodeB)
  .addEdge(START, "nodeA")
  .addEdge("nodeA", "nodeB") // B runs AFTER A, so B's updates override A's on replace fields
  .addEdge("nodeB", END)
  .compile();

async function main() {
  console.log("=".repeat(60));
  console.log("Lab 2-D: Merge Behavior – Two Nodes, Same Fields");
  console.log("=".repeat(60));

  const result = await graph.invoke({});

  console.log("\nFINAL STATE:");
  console.log("─".repeat(40));
  console.log("latestStatus :", result.latestStatus);
  // Expected: "updated by B" (nodeB ran last, replace reducer)

  console.log("log          :", result.log);
  // Expected: ["nodeA ran", "nodeB ran"] (append reducer, both kept)

  console.log("totalVotes  :", result.totalVotes);
  // Expected: 10 (3 from nodeA + 7 from nodeB, sum reducer)

  console.log("\nINSIGHT:");
  console.log("- Replace reducer: last node to run wins (nodeB overrode nodeA)");
  console.log("- Append reducer: all contributions preserved in order");
  console.log("- Sum reducer: values combined mathematically");
}

main().catch(console.error);

```

Run it:

```
npm run run src/chapter2/lab2d-merge-behavior.ts
```

2.10 Knowledge Check — Chapter 2

Summary

- **State** is the shared whiteboard that all nodes read from and write to.
- Define state using `Annotation.Root({ field: Annotation<Type>({ reducer, default }) })`.
- **Reducers** define how new values are merged into existing ones. The three main patterns are replace (last-write-wins),

append (array accumulation), and sum (numeric accumulation).

- **MessagesAnnotation** is a built-in state for chatbots, with a smart reducer that handles conversation history.
- **Channels** are the internal mechanism behind each state field — one channel per field.

Key Takeaways

1. Every field in your state needs its own reducer. Choose the reducer based on how that field should evolve.
2. Nodes receive the full state, return a partial update — only changed fields.
3. The `default` function provides the initial value for each field.
4. Use `typeof YourState.State` in node function signatures for full TypeScript type safety.
5. `MessagesAnnotation` removes boilerplate for chat-based graphs.

Practice Questions

1. What is the difference between a replace reducer and an append reducer? Give a real use case for each.
2. If two nodes run in sequence and both return `{ score: 10 }` with a sum reducer, what is the final score?
3. Why does `default` take a function `( ) => []` rather than a plain value `( [] )`?
4. What TypeScript type does `typeof MyState.State` give you?
5. When would you use `MessagesAnnotation` vs. defining your own custom state?

Mini Exercises

Exercise 1: Add a `uniqueSkills: Set<string>` field to a state schema. Write a reducer that merges two sets (hint: use `new Set([...existing, ...update])`). Does TypeScript allow `Set<string>` as an annotation type? What workaround would you use?

Exercise 2: Create a state with a `retryCount` field. Build a graph where each node increments it by 1. After 3 nodes, verify the final count is 3.

Exercise 3: Build a two-node graph where both nodes write to a `notes: string[]` field. Write a reducer that sorts the final array alphabetically.

Chapter 3 — Nodes & Edges

3.1 StateGraph: The Container for Your Workflow

`StateGraph` is the class that holds your entire graph definition. Think of it as the architect's blueprint — before any actual work is done, you describe on paper what rooms (nodes) exist and how the corridors (edges) connect them.

Creating a StateGraph:

```
import { StateGraph } from "@langchain/langgraph";

const graph = new StateGraph(YourStateAnnotation);
```

The only required argument is your state schema (the `Annotation.Root(...)` you defined). This tells the graph what shape of data to expect and how to manage it.

Important characteristics of StateGraph:

1. **Mutable before compile, immutable after.** You can call `.addNode()` and `.addEdge()` as many times as you want before calling `.compile()`. Once compiled, the graph is locked — you cannot add or modify nodes.
2. **Fluent API.** Every method on `StateGraph` returns the graph itself, so you can chain method calls. This is why you often see:

```
new StateGraph(State)
  .addNode("a", funcA)
  .addNode("b", funcB)
  .addEdge(START, "a")
  .compile()
```

3. **Named nodes.** Each node is registered with a string name. These names appear in debugging output, error messages, and streaming output. Choose descriptive names.
-

3.2 Nodes: The Workers in Your Graph

A node is a function that represents one unit of work in your workflow. It is the place where things actually happen — LLM calls, database queries, data transformations, API calls, or any other computation.

Node function signature:

```
async function myNode(  
  state: typeof MyState.State,           // current full state  
  config?: RunnableConfig                // optional runtime config (thread IDs, callbacks, etc.)  
) : Promise<Partial<typeof MyState.State>> {  
  // Do work here  
  return { someField: "new value" }; // return only changed fields  
}
```

Rules for node functions:

1. **Always async.** Even if your node does no async work, declaring it `async` is best practice and causes no harm.
2. **First argument is the full state.** You receive the complete state object with all fields. You can read any field.
3. **Return only changed fields.** You do not need to return the entire state — only the fields you want to update. LangGraph merges your return value with the existing state using reducers.
4. **Cannot directly throw to stop the graph.** If you want to handle errors gracefully within the graph, return an error flag in state. You can throw for unrecoverable failures.
5. **No side effects on the state argument.** Do not mutate `state` directly. Always return new values.

Registering a node:

```
graph.addNode("nodeName", nodeFunction);
```

The first argument is the name (a string used to reference this node in edges). The second argument is the function.

3.3 Edges: The Roads Between Nodes

Edges define the execution order. Without edges, LangGraph would not know which node to run after which.

Normal (unconditional) edges:

```
graph.addEdge("nodeA", "nodeB");
```

This means: “After `nodeA` finishes, always go to `nodeB`.” There is no condition. As soon as `nodeA` returns, `nodeB` starts.

Rules:

- You can have multiple outgoing edges from one node to run nodes in parallel (covered in Phase 2).
- A node with no outgoing edge will cause an error at runtime — always connect nodes to END or another node.
- You cannot have circular edges without an exit condition (LangGraph will enforce a recursion limit — covered in Phase 2).

Conditional (dynamic) edges — preview, covered fully in Phase 2:

```
graph.addConditionalEdges("nodeA", routingFunction);
```

Where `routingFunction` takes the state and returns the name of the next node. This enables branching.

3.4 START and END: The Entry and Exit Points

Every graph needs to know where execution begins and where it ends.

START:

`START` is a virtual node that represents the entry point. It is not a real function — it is a sentinel value that tells LangGraph “this is where the graph starts.” You always add an edge from `START` to your first real node.

```
import { START } from "@langchain/langgraph";
```

```
graph.addEdge(START, "firstNode");
```

END:

`END` is a virtual node that represents the exit point. When a node has an edge to `END`, reaching that node terminates graph execution and returns the final state to the caller.

```
import { END } from "@langchain/langgraph";

graph.addEdge("lastNode", END);
```

Multiple paths to END:

A graph can have multiple nodes that connect to `END`. This is common in branched workflows — both the “approve” path and the “reject” path might ultimately connect to `END`.

Why are START and END needed?

Other workflow frameworks often assume there is exactly one entry node and one exit node. LangGraph makes this explicit and flexible. A graph could technically have multiple entry points (though this is rare) or multiple exit paths.

3.5 Compiling the Graph

`.compile()` is the final step before you can run a graph. It performs several important tasks:

1. **Validation.** Checks that all nodes referenced in edges actually exist. Detects disconnected nodes, missing edges, and other structural issues.
2. **Optimization.** Analyses the graph structure to determine execution order and identify opportunities for parallel execution.
3. **Locks the definition.** After compiling, you cannot add or modify nodes or edges.
4. **Optionally accepts a checkpointer.** This is how you add persistence (covered in Phase 3):

```
const graph = workflow.compile({ checkpointer: memorySaver });
```

What `.compile()` returns:

A `CompiledStateGraph` object — this is what you actually call `.invoke()`, `.stream()`, and `.batch()` on.

```
const compiled = workflow.compile();
// compiled is a CompiledStateGraph
const result = await compiled.invoke({ ... });
```

3.6 Running the Graph: `invoke()`, `stream()`, and `batch()`

Once compiled, a graph can be run in three modes:

`invoke(input, config?)` — Run and return final state

```
const finalState = await graph.invoke({ message: "Hello" });
```

- Runs the entire graph synchronously (awaited)
- Returns the complete final state after all nodes have finished
- Use this when you need the final result and do not care about intermediate steps

`stream(input, config?)` — Yield output after each node

```
const stream = await graph.stream({ message: "Hello" });

for await (const chunk of stream) {
  // chunk is an object: { nodeName: { ...partialState } }
  const [nodeName, output] = Object.entries(chunk)[0];
```



```

    console.log(`Node "${nodeName}" output:`, output);
  }

```

- Returns an async iterable
- Yields one object per node that runs, containing the node's name and its partial state output
- Use this for real-time feedback, progress indicators, or when you want to react to intermediate results

Stream modes:

`stream()` can be called with different modes to control what data it yields:

```

// Default: yields partial node outputs
const stream = await graph.stream(input);

// "values" mode: yields the FULL state after every node
const stream = await graph.stream(input, { streamMode: "values" });

// "updates" mode: same as default (partial outputs)
const stream = await graph.stream(input, { streamMode: "updates" });

```

The "values" mode is useful when you want to see the cumulative state rather than just what each node changed.

`batch(inputs[], config?)` — Run multiple inputs in parallel

```

const results = await graph.batch([
  { message: "Input 1" },
  { message: "Input 2" },
  { message: "Input 3" },
]);
// results is an array of final states, one per input

```

- Runs the graph independently for each input, in parallel
- Returns an array of final states
- Use this for bulk processing (analyzing multiple documents, scoring multiple candidates)

3.7 Lab 3-A: Build a 3-Node Linear Graph and Compile It

Create the file:

```

mkdir -p src/chapter3
touch src/chapter3/lab3a-three-node-graph.ts

```

```

// src/chapter3/lab3a-three-node-graph.ts

import * as dotenv from "dotenv";
dotenv.config();

import { StateGraph, START, END, Annotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0.3 });

// =====
// SCENARIO: A 3-step content pipeline
// Node 1 (research) - gather key facts about a topic
// Node 2 (draft) - write a short paragraph
// Node 3 (title) - generate a catchy title
// =====
const ContentState = Annotation.Root({
  topic: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "",
  }),
  keyFacts: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "",
  }),
  paragraph: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "",
  }),
  title: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "",
  })
});

```

```

    }},
    // Track which nodes have completed (for observability)
    executionLog: Annotation<string[]>({
      reducer: (e: string[], u: string[]) => [...e, ...u],
      default: () => [],
    }),
  });

// =====
// NODE 1: RESEARCH
// =====
async function researchNode(
  state: typeof ContentState.State
): Promise<Partial<typeof ContentState.State>> {
  console.log(`\n[Node 1: research] Topic: "${state.topic}"`);

  const res = await llm.invoke([
    new SystemMessage("List 4 interesting facts about the given topic. Number them 1-4. Be concise."),
    new HumanMessage(state.topic),
  ]);

  return {
    keyFacts: res.content as string,
    executionLog: ["research completed"],
  };
}

// =====
// NODE 2: DRAFT
// =====
async function draftNode(
  state: typeof ContentState.State
): Promise<Partial<typeof ContentState.State>> {
  console.log(`\n[Node 2: draft] Writing paragraph from facts...`);

  const res = await llm.invoke([
    new SystemMessage("Write one concise, engaging paragraph using these facts. No bullet points – flowing prose only."),
    new HumanMessage(`Topic: ${state.topic}\nFacts:\n${state.keyFacts}`),
  ]);

  return {
    paragraph: res.content as string,
    executionLog: ["draft completed"],
  };
}

// =====
// NODE 3: TITLE GENERATION
// =====
async function titleNode(
  state: typeof ContentState.State
): Promise<Partial<typeof ContentState.State>> {
  console.log(`\n[Node 3: title] Generating title...`);

  const res = await llm.invoke([
    new SystemMessage("Create a short, catchy article title (max 8 words) for this paragraph. Return ONLY the title."),
    new HumanMessage(state.paragraph),
  ]);

  return {
    title: res.content as string,
    executionLog: ["title completed"],
  };
}

// =====
// BUILD AND COMPILE THE GRAPH
// =====
// Linear flow: START → research → draft → title → END
// =====
const workflow = new StateGraph(ContentState)
  .addNode("research", researchNode)
  .addNode("draft", draftNode)
  .addNode("title", titleNode)
  .addEdge(START, "research")
  .addEdge("research", "draft")
  .addEdge("draft", "title")
  .addEdge("title", END);

// compile() accepts an optional config object.
// For now we compile with no options – persistence is added in Phase 3.
const graph = workflow.compile();

// =====
// HOW TO INSPECT A COMPILED GRAPH'S STRUCTURE
// You can get a Mermaid diagram string to visualise the graph
// =====
function printGraphStructure() {
  console.log("\n[Graph Structure]");
  // graph.getGraph() returns a DrawableGraph with a drawMermaid() method

```

```

    try {
      const mermaid = graph.getGraph().drawMermaid();
      console.log("Mermaid diagram:\n");
      console.log(mermaid);
    } catch {
      console.log("(drawMermaid not available in this environment)");
    }
  }
}

async function main() {
  console.log("=".repeat(60));
  console.log("Lab 3-A: 3-Node Linear Graph");
  console.log("=".repeat(60));

  printGraphStructure();

  const result = await graph.invoke({
    topic: "The history of the internet",
  });

  console.log("\n" + "=".repeat(60));
  console.log("FINAL CONTENT:");
  console.log("─".repeat(60));
  console.log("TITLE      :", result.title);
  console.log("─".repeat(60));
  console.log("PARAGRAPH :");
  console.log(result.paragraph);
  console.log("─".repeat(60));
  console.log("KEY FACTS :");
  console.log(result.keyFacts);
  console.log("─".repeat(60));
  console.log("EXECUTION LOG:", result.executionLog);
  console.log("=".repeat(60));
}

main().catch(console.error);

```

Run it:

```
npm run run src/chapter3/lab3a-three-node-graph.ts
```

3.8 Lab 3-B: Add a Node That Mutates Only Part of the State

This lab reinforces the concept that nodes only return what they change.

Create the file:

```
touch src/chapter3/lab3b-partial-updates.ts
```

```

// src/chapter3/lab3b-partial-updates.ts
// Demonstrates partial state updates and that untouched fields are preserved

import { StateGraph, START, END, Annotation } from "@langchain/langgraph";

const OrderState = Annotation.Root({
  orderId: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "",
  }),
  customerName: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "",
  }),
  items: Annotation<string[]>({
    reducer: (e: string[], u: string[]) => [...e, ...u],
    default: () => [],
  }),
  totalAmount: Annotation<number>({
    reducer: (_e: number, u: number) => u,
    default: () => 0,
  }),
  status: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "new",
  }),
  trackingNumber: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "",
  }),
});

// This node only updates `status` and `totalAmount`.

```

```

// It does NOT touch orderId, customerName, items, or trackingNumber.
// Those fields remain exactly as they were.
async function validateOrderNode(
  state: typeof OrderState.State
): Promise<Partial<typeof OrderState.State>> {
  console.log("\n[validateOrder] Validating order:", state.orderId);
  console.log("[validateOrder] Reading items:", state.items);
  console.log("[validateOrder] NOT modifying: orderId, customerName, trackingNumber");

  const total = state.items.length * 29.99; // simple mock pricing

  // Only return changed fields
  return {
    status: "validated",
    totalAmount: total,
    // orderId, customerName, items, trackingNumber are NOT included here
    // They will remain unchanged in state
  };
}

// This node only updates `status` and `trackingNumber`.
// It reads totalAmount (set by previous node) but does not modify it.
async function shipOrderNode(
  state: typeof OrderState.State
): Promise<Partial<typeof OrderState.State>> {
  console.log("\n[shipOrder] Processing shipment. Total:", state.totalAmount);
  console.log("[shipOrder] Customer:", state.customerName);

  const trackingNumber = `TRK-${state.orderId}-${Date.now().toString().slice(-6)}`;

  return {
    status: "shipped",
    trackingNumber,
    // everything else untouched
  };
}

const graph = new StateGraph(OrderState)
  .addNode("validate", validateOrderNode)
  .addNode("ship", shipOrderNode)
  .addEdge(START, "validate")
  .addEdge("validate", "ship")
  .addEdge("ship", END)
  .compile();

async function main() {
  console.log("=".repeat(60));
  console.log("Lab 3-B: Partial State Updates");
  console.log("=".repeat(60));

  const initialOrder = {
    orderId: "ORD-001",
    customerName: "Alice Johnson",
    items: ["TypeScript Book", "LangGraph Course", "Keyboard"],
    // totalAmount, status, trackingNumber use their defaults (0, "new", "")
  };

  console.log("\nInitial order:", initialOrder);

  const result = await graph.invoke(initialOrder);

  console.log("\n" + "=".repeat(60));
  console.log("FINAL STATE (all fields):");
  console.log("=".repeat(60));
  console.log("orderId      :", result.orderId);           // unchanged from input
  console.log("customerName  :", result.customerName);        // unchanged from input
  console.log("items         :", result.items);                  // unchanged from input
  console.log("totalAmount   :", result.totalAmount);            // set by validateOrder
  console.log("status        :", result.status);                  // set by shipOrder (overwrote validate's)
  console.log("trackingNumber:", result.trackingNumber);          // set by shipOrder
  console.log("=".repeat(60));
  console.log("\nKEY INSIGHT: Fields not returned by a node are preserved exactly.");
}

main().catch(console.error);

```

Run it:

```
npm run run src/chapter3/lab3b-partial-updates.ts
```

3.9 Lab 3-C: invoke() and Inspect Final State

This lab explores `invoke()` in depth — including how the initial input merges with defaults.

Create the file:

```
touch src/chapter3/lab3c-invoke-inspect.ts
```

```
// src/chapter3/lab3c-invoke-inspect.ts

import * as dotenv from "dotenv";
dotenv.config();

import { StateGraph, START, END, Annotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

const AnalysisState = Annotation.Root({
  userQuestion: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "",
  }),
  category: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "unknown",
  }),
  confidence: Annotation<number>({
    reducer: (_e: number, u: number) => u,
    default: () => 0,
  }),
  answer: Annotation<string>({
    reducer: (_e: string, u: string) => u,
    default: () => "",
  }),
  processingTimeMs: Annotation<number>({
    reducer: (e: number, u: number) => e + u,
    default: () => 0,
  }),
});

async function classifyNode(state: typeof AnalysisState.State) {
  const start = Date.now();
  const res = await llm.invoke([
    new SystemMessage(
      "Classify the question into one category. " +
      "Return a JSON object with ONLY these fields: {\"category\": \"science|history|math|other\", \"confidence\": 0.0-1.0}. " +
      "Return ONLY valid JSON, no markdown."
    ),
    new HumanMessage(state.userQuestion),
  ]);

  const parsed = JSON.parse(res.content as string);
  return {
    category: parsed.category as string,
    confidence: Math.round(parsed.confidence * 100),
    processingTimeMs: Date.now() - start,
  };
}

async function answerNode(state: typeof AnalysisState.State) {
  const start = Date.now();
  const res = await llm.invoke([
    new SystemMessage(
      `You are answering a ${state.category} question. ` +
      "Provide a clear, concise answer in 2-3 sentences."
    ),
    new HumanMessage(state.userQuestion),
  ]);
  return {
    answer: res.content as string,
    processingTimeMs: Date.now() - start,
  };
}

const graph = new StateGraph(AnalysisState)
  .addNode("classify", classifyNode)
  .addNode("answer", answerNode)
  .addEdge(START, "classify")
  .addEdge("classify", "answer")
  .addEdge("answer", END)
  .compile();

// =====
// DEMONSTRATE DIFFERENT WAYS TO USE invoke()
// =====

async function main() {
  console.log("=".repeat(60));
  console.log("Lab 3-C: invoke() – Input Merging and State Inspection");
  console.log("=".repeat(60));
}
```

```
// — Run 1: Minimal input – let defaults fill in the rest
console.log("\n[Run 1] Only providing userQuestion. Category, confidence, answer use defaults.");
const result1 = await graph.invoke({
  userQuestion: "Why is the sky blue?",
});
console.log("\nRun 1 Result:");
console.log("  question      :", result1.userQuestion);
console.log("  category      :", result1.category);
console.log("  confidence (%) :", result1.confidence);
console.log("  processingTimeMs:", result1.processingTimeMs, "ms");
console.log("  answer        :", result1.answer);

// — Run 2: Override a default value in the initial input
// You can provide initial values even for fields that have defaults.
// This is useful for pre-seeding state.
console.log("\n[Run 2] Providing initial category to skip classification logic...");
const result2 = await graph.invoke({
  userQuestion: "What is 2 + 2?",
  category: "math",           // pre-set so node can read it
  confidence: 100,           // pre-set
});
console.log("\nRun 2 Result:");
console.log("  category (after nodes ran):", result2.category); // may be overwritten by classify
console.log("  answer:", result2.answer);

// — Run 3: Access specific fields from the result
console.log("\n[Run 3] Destructuring only needed fields from result...");
const { answer, confidence, processingTimeMs } = await graph.invoke({
  userQuestion: "When was the Eiffel Tower built?",
});
console.log("\nRun 3 – Destructured result:");
console.log("  Answer      :", answer);
console.log("  Confidence   :", confidence, "%");
console.log("  Time taken   :", processingTimeMs, "ms");
}

main().catch(console.error);
```

Run it:

```
npm run run src/chapter3/lab3c-invoke-inspect.ts
```

3.10 Lab 3-D: stream() Intermediate States

This is the most important lab in Chapter 3. Streaming is what makes LangGraph's execution transparent.

Create the file:

```
touch src/chapter3/lab3d-streaming.ts
```

```
// src/chapter3/lab3d-streaming.ts
```

```
import * as dotenv from "dotenv";
dotenv.config();
```

```
import { StateGraph, START, END, Annotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";
```

```
const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0.5 });
```

```
const StoryState = Annotation.Root({
  prompt: Annotation<string>({ reducer: (_e, u) => u, default: () => "" }),
  setting: Annotation<string>({ reducer: (_e, u) => u, default: () => "" }),
  characters: Annotation<string>({ reducer: (_e, u) => u, default: () => "" }),
  conflict: Annotation<string>({ reducer: (_e, u) => u, default: () => "" }),
  story: Annotation<string>({ reducer: (_e, u) => u, default: () => "" }),
});
```

```
async function createSetting(state: typeof StoryState.State) {
  const res = await llm.invoke([
    new SystemMessage("Create a vivid story setting in 1-2 sentences. Return ONLY the setting description."),
    new HumanMessage(`Story prompt: ${state.prompt}`),
  ]);
  return { setting: res.content as string };
}
```

```
async function createCharacters(state: typeof StoryState.State) {
  const res = await llm.invoke([
    new SystemMessage("Create 2 characters for a story. Name and one-sentence description each. Return ONLY the character descriptions."),
    new HumanMessage(`Setting: ${state.setting}\nPrompt: ${state.prompt}`),
  ]);
}
```

```

    });
    return { characters: res.content as string };
}

async function createConflict(state: typeof StoryState.State) {
    const res = await llm.invoke([
        new SystemMessage("Create a central conflict for the story in 1-2 sentences. Return ONLY the conflict description."),
        new HumanMessage(`Setting: ${state.setting}\nCharacters: ${state.characters}`),
    ]);
    return { conflict: res.content as string };
}

async function writeStory(state: typeof StoryState.State) {
    const res = await llm.invoke([
        new SystemMessage("Write a short 3-paragraph story using the provided elements. Engaging and creative."),
        new HumanMessage(
            `Setting: ${state.setting}\n` +
            `Characters: ${state.characters}\n` +
            `Conflict: ${state.conflict}`
        ),
    ]);
    return { story: res.content as string };
}

const storyGraph = new StateGraph(StoryState)
    .addNode("setting", createSetting)
    .addNode("characters", createCharacters)
    .addNode("conflict", createConflict)
    .addNode("writeStory", writeStory)
    .addEdge(START, "setting")
    .addEdge("setting", "characters")
    .addEdge("characters", "conflict")
    .addEdge("conflict", "writeStory")
    .addEdge("writeStory", END)
    .compile();

async function main() {
    console.log("=".repeat(60));
    console.log("Lab 3-D: Streaming Graph Execution");
    console.log("=".repeat(60));

    const startTime = Date.now();

    // -----
    // MODE 1: DEFAULT STREAM – yields partial node outputs (delta mode)
    // Each chunk tells you which node just finished and what it returned.
    // -----
    console.log("\n— MODE 1: Default stream (partial node outputs) —\n");

    const defaultStream = await storyGraph.stream({
        prompt: "A librarian discovers a book that predicts the future",
    });

    for await (const chunk of defaultStream) {
        const [nodeName, output] = Object.entries(chunk)[0] as [string, Record<string, unknown>];
        const elapsed = ((Date.now() - startTime) / 1000).toFixed(1);

        console.log(`[${elapsed}s] Node "${nodeName}" completed`);

        // Print the field that this node returned
        const outputKeys = Object.keys(output);
        for (const key of outputKeys) {
            const value = output[key] as string;
            const preview = value.length > 80 ? value.substring(0, 80) + "...": value;
            console.log(` → ${key}: "${preview}"`);
        }
        console.log();
    }

    // -----
    // MODE 2: "values" STREAM – yields full cumulative state after each node
    // Use this when you need to see the complete state at each step.
    // -----
    console.log("=".repeat(60));
    console.log("— MODE 2: 'values' stream (full state after each node) —\n");

    const valuesStream = await storyGraph.stream(
        { prompt: "A robot learns to dream" },
        { streamMode: "values" } // <- the key difference
    );

    let stepCount = 0;
    for await (const state of valuesStream) {
        // In "values" mode, each chunk IS the full state object
        stepCount++;
        console.log(`[Step ${stepCount}] Full state snapshot:`);

        const populatedFields = Object.entries(state as Record<string, unknown>)
            .filter(([v]) => v !== "" && v !== 0 && v !== false)
            .map(([k]) => k);
    }

```

```

    console.log(`  Populated fields: [${populatedFields.join(", ")}]`);

    // Show which field was just populated (the one added in this step)
    if (stepCount <= 4) {
      const fieldNames = ["setting", "characters", "conflict", "story"];
      const justAdded = fieldNames[stepCount - 1];
      if (justAdded && (state as Record<string, unknown>)[justAdded]) {
        const val = (state as Record<string, unknown>)[justAdded] as string;
        console.log(`  "${justAdded}" just added: "${val.substring(0, 60)}..."`);
      }
    }
    console.log();
  }

  console.log("=".repeat(60));
  console.log("Streaming complete. Total time:", ((Date.now() - startTime) / 1000).toFixed(1), "seconds");
  console.log("=".repeat(60));
}

main().catch(console.error);

```

Run it:

```
npm run run src/chapter3/lab3d-streaming.ts
```

What to observe: - In Mode 1 (default), each chunk only contains what the node returned — the delta - In Mode 2 (values), each chunk is the complete accumulated state — you can see fields filling in one by one - The timestamps show you real-time progress as each LLM call completes

3.11 Knowledge Check — Chapter 3

Summary

- `StateGraph(StateAnnotation)` is the container that holds your graph definition before compilation.
- Nodes are async functions: `(state) => partialStateUpdate` .
- `addEdge(from, to)` creates unconditional connections between nodes.
- `START` and `END` are virtual entry and exit points — always add edges to/from them.
- `.compile()` validates, optimises, and locks the graph definition.
- `invoke()` returns the final state. `stream()` yields intermediate outputs as nodes complete. `batch()` processes multiple inputs in parallel.

Key Takeaways

1. The graph is defined with method chaining: `new StateGraph() → addNode() × N → addEdge() × N → compile()` .
2. `compile()` is mandatory before `invoke()` or `stream()` .
3. `stream()` with default mode yields `{ nodeName: partialOutput }` . With `{ streamMode: "values" }` it yields full state.
4. Node names are string identifiers — choose descriptive names because they appear in logs and errors.
5. A node that has no outgoing edge will cause a runtime error. Always connect all nodes to `END` or another node.

Practice Questions

1. What happens if you call `invoke()` on a graph that has not been compiled?
2. What is the difference between `stream()` in default mode versus `{ streamMode: "values" }` mode?
3. Can you add a node to a graph after calling `.compile()` ? Why or why not?
4. How does `batch()` differ from calling `invoke()` multiple times?
5. If a node function is not declared `async` , can it still be used in a `LangGraph`?

Mini Exercise

Create a graph with 4 nodes: 1. `generateNumbers` — returns `{ numbers: [1, 5, 3, 9, 2, 7] }` (hardcoded) 2. `sortNumbers` — sorts the array and returns the sorted version 3. `calculateStats` — computes min, max, and sum from the sorted array, stores them in separate state fields 4. `formatReport` — builds a string report from the statistics

Run it with `stream()` and observe each step. Add appropriate state fields and reducers.

Phase 1 Capstone Project

Build an AI-Powered Article Quality Checker

This project combines everything from Phase 1 into a real, useful application. It takes an article draft as input, runs it through a multi-node analysis pipeline, and produces a structured quality report with specific, actionable feedback.

What this project demonstrates: - Rich multi-field state schema with multiple reducer types - 5-node linear graph with clear responsibility separation - LLM integration across multiple nodes - State accumulation (feedback notes building up across nodes) - Streaming for real-time progress display - Full invoke for final result retrieval

Create the project file:

```
touch src/chapter3/capstone-article-checker.ts
```

```
// src/chapter3/capstone-article-checker.ts
// PHASE 1 CAPSTONE PROJECT: AI Article Quality Checker

import * as dotenv from "dotenv";
dotenv.config();

import { StateGraph, START, END, Annotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0.2 });

// -----
// STATE SCHEMA
//
// All analysis results accumulate through the nodes.
// feedbackItems uses APPEND so every node can contribute feedback.
// scores use last-write-wins (each node writes its own score domain).
// -----
const ArticleState = Annotation.Root({
  // INPUT (set once, never changes)
  articleText: Annotation<string>({
    reducer: (_e, u) => u,
    default: () => "",
  }),
  targetAudience: Annotation<string>({
    reducer: (_e, u) => u,
    default: () => "general",
  }),

  // NODE 1 OUTPUT: basic metadata
  wordCount: Annotation<number>({
    reducer: (_e, u) => u,
    default: () => 0,
  }),
  sentenceCount: Annotation<number>({
    reducer: (_e, u) => u,
    default: () => 0,
  }),
  avgWordsPerSentence: Annotation<number>({
    reducer: (_e, u) => u,
    default: () => 0,
  }),

  // NODE 2 OUTPUT: readability analysis
  readabilityScore: Annotation<number>({
    reducer: (_e, u) => u,
    default: () => 0,
  }),
  readabilityLevel: Annotation<string>({
    reducer: (_e, u) => u,
    default: () => "",
  }),

  // NODE 3 OUTPUT: structure analysis
  hasIntroduction: Annotation<boolean>({
    reducer: (_e, u) => u,
    default: () => false,
  }),
  hasConclusion: Annotation<boolean>({
    reducer: (_e, u) => u,
    default: () => false,
  }),
  structureScore: Annotation<number>({
    reducer: (_e, u) => u,
    default: () => 0,
  })
});
```

```

    }},

    // NODE 4 OUTPUT: grammar and style
    grammarScore: Annotation<number>({
      reducer: (_e, u) => u,
      default: () => 0,
    }),
    styleIssues: Annotation<string[]>({
      reducer: (e, u) => [...e, ...u],
      default: () => [],
    }),

    // ACCUMULATES across ALL nodes – each node appends its findings
    feedbackItems: Annotation<string[]>({
      reducer: (e, u) => [...e, ...u],
      default: () => [],
    }),

    // NODE 5 OUTPUT: final report
    overallScore: Annotation<number>({
      reducer: (_e, u) => u,
      default: () => 0,
    }),
    finalReport: Annotation<string>({
      reducer: (_e, u) => u,
      default: () => "",
    }),
  });

  // _____
  // NODE 1: BASIC METRICS (no LLM needed – pure string processing)
  // _____
  async function basicMetricsNode(
    state: typeof ArticleState.State
  ): Promise<Partial<typeof ArticleState.State>> {
    console.log(`\n[Node 1: basicMetrics] Counting words and sentences...`);

    const words = state.articleText.trim().split(/\s+/).filter(Boolean);
    const sentences = state.articleText
      .split(/[\.!?]+/)
      .map((s) => s.trim())
      .filter((s) => s.length > 0);

    const wordCount = words.length;
    const sentenceCount = sentences.length;
    const avgWordsPerSentence = sentenceCount > 0
      ? Math.round(wordCount / sentenceCount)
      : 0;

    const feedback: string[] = [];
    if (wordCount < 300) feedback.push("Article is short (under 300 words). Consider expanding.");
    if (avgWordsPerSentence > 25) feedback.push("Sentences are very long (avg " + avgWordsPerSentence + " words). Consider shorter sentences.");
    if (avgWordsPerSentence < 10) feedback.push("Sentences may be too short (avg " + avgWordsPerSentence + " words). Consider more detail.")

    return { wordCount, sentenceCount, avgWordsPerSentence, feedbackItems: feedback };
  }

  // _____
  // NODE 2: READABILITY ANALYSIS (LLM)
  // _____
  async function readabilityNode(
    state: typeof ArticleState.State
  ): Promise<Partial<typeof ArticleState.State>> {
    console.log(`\n[Node 2: readability] Analysing readability...`);

    const res = await llm.invoke([
      new SystemMessage(
        "Analyse the readability of this article for a " + state.targetAudience + " audience. " +
        "Return ONLY a JSON object: {\n\"score\": 0-100, \n\"level\": \"Elementary|Intermediate|Advanced\", \n\"feedback\": \"one sentence about readability\"}. " +
        "No markdown, no explanation."
      ),
      new HumanMessage(
        `Word count: ${state.wordCount}. Avg words/sentence: ${state.avgWordsPerSentence}.\n\nArticle:\n${state.articleText.substring(0, 1000)}`
      ),
    ]),
  };

  const parsed = JSON.parse(res.content as string);
  return {
    readabilityScore: parsed.score,
    readabilityLevel: parsed.level,
    feedbackItems: [parsed.feedback],
  };
}

// _____
// NODE 3: STRUCTURE ANALYSIS (LLM)
// _____

```

```
async function structureNode(
  state: typeof ArticleState.State
): Promise<Partial<typeof ArticleState.State>> {
  console.log(`\n[Node 3: structure] Checking article structure...`);

  const res = await llm.invoke([
    new SystemMessage(
      "Analyse the structure of this article. " +
      "Return ONLY a JSON object: {\n\"hasIntroduction\": boolean, \n\"hasConclusion\": boolean, \n\"score\": 0-100, \n\"feedback\": \"one\nsentence\". " +
      "No markdown."
    ),
    new HumanMessage(state.articleText),
  ]);

  const parsed = JSON.parse(res.content as string);
  return {
    hasIntroduction: parsed.hasIntroduction,
    hasConclusion: parsed.hasConclusion,
    structureScore: parsed.score,
    feedbackItems: [parsed.feedback],
  };
}

// _____
// NODE 4: GRAMMAR AND STYLE (LLM)
// _____
async function grammarNode(
  state: typeof ArticleState.State
): Promise<Partial<typeof ArticleState.State>> {
  console.log(`\n[Node 4: grammar] Checking grammar and style...`);

  const res = await llm.invoke([
    new SystemMessage(
      "Review grammar and writing style. " +
      "Return ONLY a JSON object: {\n\"score\": 0-100, \n\"issues\": [\n\"issue 1\", \n\"issue 2\"], \n\"feedback\": \"one positive sentence\". " +
      "List up to 3 specific issues. No markdown."
    ),
    new HumanMessage(state.articleText),
  ]);

  const parsed = JSON.parse(res.content as string);
  return {
    grammarScore: parsed.score,
    styleIssues: parsed.issues || [],
    feedbackItems: [parsed.feedback, ...(parsed.issues || []).map((i: string) => "Grammar: " + i)],
  };
}

// _____
// NODE 5: FINAL REPORT GENERATION
// _____
// This node reads ALL state accumulated by previous nodes and produces
// a comprehensive report. This is where the "shared whiteboard" pattern
// shows its full power.
// _____
async function reportNode(
  state: typeof ArticleState.State
): Promise<Partial<typeof ArticleState.State>> {
  console.log(`\n[Node 5: report] Generating final report...`);

  const overallScore = Math.round(
    (state.readabilityScore + state.structureScore + state.grammarScore) / 3
  );

  const report = `
```

ARTICLE QUALITY REPORT

OVERVIEW	
Target Audience	: \${state.targetAudience}
Word Count	: \${state.wordCount} words
Sentence Count	: \${state.sentenceCount} sentences
Avg Sentence Len	: \${state.avgWordsPerSentence} words/sentence
SCORES	
Readability	: \${state.readabilityScore}/100 (\${state.readabilityLevel})
Structure	: \${state.structureScore}/100
Grammar & Style	: \${state.grammarScore}/100
OVERALL SCORE	: \${overallScore}/100
STRUCTURE CHECKLIST	
Has Introduction	: \${state.hasIntroduction ? "YES" : "NO"}
Has Conclusion	: \${state.hasConclusion ? "YES" : "NO"}

```
SPECIFIC FEEDBACK (${state.feedbackItems.length} items)

${state.feedbackItems.map((f, i) => `${i + 1}. ${f}`).join("\n")}
```

VERDICT

```
${overallScore >= 80
  ? "EXCELLENT – This article meets high quality standards."
  : overallScore >= 60
  ? "GOOD – Minor improvements recommended. Review feedback above."
  : "NEEDS WORK – Significant improvements required before publishing."}
.trim();

return { overallScore, finalReport: report };
}
```

```
//
// BUILD THE GRAPH
//
```

```
const qualityChecker = new StateGraph(ArticleState)
  .addNode("basicMetrics", basicMetricsNode)
  .addNode("readability", readabilityNode)
  .addNode("structure", structureNode)
  .addNode("grammar", grammarNode)
  .addNode("report", reportNode)
  .addEdge(START, "basicMetrics")
  .addEdge("basicMetrics", "readability")
  .addEdge("readability", "structure")
  .addEdge("structure", "grammar")
  .addEdge("grammar", "report")
  .addEdge("report", END)
  .compile();
```

```
//
// TEST ARTICLE
//
```

```
const sampleArticle = `
The Rise of Artificial Intelligence in Everyday Life
```

Artificial intelligence is no longer a futuristic concept reserved for science fiction. Today, AI systems are woven into the fabric of our daily routines in ways both visible and invisible.

When you unlock your phone with your face, ask a voice assistant for the weather, or receive a product recommendation while shopping online, you are interacting with AI. These systems analyze patterns in enormous datasets to make predictions and decisions with remarkable accuracy.

Healthcare is one of the most promising frontiers for AI adoption. Machine learning algorithms can now detect early signs of cancer in medical images with accuracy matching or exceeding that of trained specialists. This technology has the potential to save countless lives by enabling earlier intervention.

However, the rapid advancement of AI also raises important questions. Issues of privacy, bias in algorithmic decision-making, and the potential displacement of workers in certain industries demand careful consideration by policymakers, technologists, and society at large.

The future of AI is not predetermined. The choices we make today about how to develop, deploy, and regulate these systems will shape the kind of AI-powered world our children inherit. Thoughtful engagement from informed citizens is not merely helpful – it is essential.

```
`;

async function main() {
  console.log("=".repeat(60));
  console.log("PHASE 1 CAPSTONE: AI Article Quality Checker");
  console.log("=".repeat(60));

  // OPTION A: Stream for real-time progress
  console.log("\n--- STREAMING PROGRESS ---");
  const stream = await qualityChecker.stream({
    articleText: sampleArticle.trim(),
    targetAudience: "general public",
  });

  for await (const chunk of stream) {
    const [nodeName] = Object.entries(chunk)[0];
    console.log(` ✓ "${nodeName}" completed`);
  }

  // OPTION B: Invoke separately for the complete final state
  console.log("\n--- RUNNING FULL ANALYSIS ---");
  const result = await qualityChecker.invoke({
    articleText: sampleArticle.trim(),
    targetAudience: "general public",
  });

  // Print the final structured report
  console.log("\n" + result.finalReport);

  // You can also access individual state fields programmatically
  console.log("\n--- PROGRAMMATIC ACCESS TO STATE ---");
  console.log("Overall Score:", result.overallScore);
  console.log("Style Issues Found:", result.styleIssues.length);
  console.log("Feedback Item Count:", result.feedbackItems.length);
}
```

```
main().catch(console.error);
```

Run it:

```
npm run run src/chapter3/capstone-article-checker.ts
```

Expected behaviour: - You see streaming progress as each of 5 nodes completes - The final report provides a comprehensive quality assessment with scores across three dimensions - Individual state fields are accessible programmatically for further processing

Extend the Capstone (Optional Challenges)

1. **Add a tone analysis node** between `grammar` and `report` that classifies the article's tone (formal, casual, academic) and adds it to the report.
 2. **Make word count warnings smarter** — implement different thresholds based on `targetAudience` (e.g., children's content should be under 500 words).
 3. **Add batch processing** — run the quality checker on an array of three different sample articles using `graph.batch()` and compare their scores.
 4. **Add streaming feedback** — modify the main function to print each `feedbackItems` update as nodes complete, rather than waiting for the final report.
-

Final Notes

What You Have Learned in Phase 1

You can now:

- Explain why LangGraph exists and when to choose it over a Chain or AgentExecutor
- Define typed state schemas using `Annotation.Root()` with appropriate reducers for each field
- Write node functions that receive state and return partial updates
- Connect nodes with edges and understand the role of `START` and `END`
- Compile a graph and run it with `invoke()`, `stream()`, and `batch()`
- Observe intermediate state changes using `stream()` with different modes
- Build realistic multi-step workflows where nodes communicate through shared state

What Comes Next in Phase 2 (Control Flow)

Phase 2 introduces: - `addConditionalEdges()` — branching based on state - Cycles and loops — routing back to earlier nodes - The `Send` API — parallel fan-out execution - The `Command` object — combining routing decisions with state updates - `recursionLimit` — preventing infinite loops

These capabilities transform a linear pipeline into a truly dynamic, adaptive workflow — the defining feature of LangGraph over simpler alternatives.

End of Phase 1 Foundation Guide